

FTEC5660 Individual Project: Reproducing GitTaskBench on a Single Task (Eparse_01)

Ruiqian Luo
Student ID: 1155244663
Course: FTEC5660
February 27, 2026

1 Project Summary and Reproduction Target

1.1 Project

This project reproduces a small, clearly-defined subset of an agent benchmark: **GitTaskBench** on one task, **Eparse_01**. GitTaskBench evaluates intelligent agents on repository-based tasks with an official grading script.

1.2 Target claim / metric

Following the assignment scope guidance (reproduce a small task/experiment), my target is:

- **Task Pass Rate** for Eparse_01 using the `gittaskbench grade` official evaluation.
- I report the official score components (filename match, content similarity) and the final status (PASSED/FAILED).

1.3 Repository link

My submission repo (artifacts + instructions):

<https://github.com/RuiqianLuo/FTEC5660-individual-project/tree/main>

Upstream benchmark repo (authors):

<https://github.com/QuantaAlpha/GitTaskBench>

2 Setup Notes (Environment, Data, Keys, Compute)

2.1 Hardware/OS

- OS: Windows 10/11
- Shells used: Git Bash + Anaconda Prompt

2.2 Python environment

I used a Conda environment named `gtb` (Python 3.10). Some components were also tested with a system Python 3.11, but the final grading run shown in the results section uses the `gtb` environment.

2.3 API keys

An LLM API key is required only when running the agent tool (Aider). **No secrets are committed to GitHub.** Keys are set as environment variables locally.

2.4 Compute budget

This reproduction is lightweight: it runs one task and a single official grading script.

3 Reproduction Target(s) and Metric Definition

3.1 Task

Task ID: `Eparse_01`.

The task expects the system to generate a prediction output text file that contains structured blocks corresponding to Excel content.

3.2 Metric

The official grader reports:

- **Process** (whether a valid output was produced)
- **Result / status** (PASSED/FAILED)
- Filename match rate
- Content similarity per file + average similarity
- Final score (weighted combination of filename match and content similarity) with a pass threshold

4 How to Reproduce (Step-by-Step)

This section is written so that a new user can replicate my final run.

4.1 Step A: Install dependencies

Run in **Anaconda Prompt** (inside env `gtb`) or in Git Bash using the same Python:

```
# In your conda env (gtb):
python -m pip install --upgrade pip
python -m pip install pandas openpyxl
```

4.2 Step B: Ensure Eparse import works (PYTHONPATH)

The script relies on a local repository under: `D:\projects\GitTaskBench\code_base\Eparse`
In Git Bash:

```
export PYTHONPATH="/d/projects/GitTaskBench/code_base/Eparse:$PYTHONPATH"
python3 -c "import _eparse;_print('import_ok:',_eparse.__file__)"
```

(If you use Anaconda Prompt / Windows CMD, set PYTHONPATH using Windows syntax.)

4.3 Step C: Generate prediction output (the file graded by GitTaskBench)

From the GitTaskBench root directory:

```
cd /d/projects/GitTaskBench
python prompt/Eparse_01/process_excel.py
```

This script reads Excel inputs from:

```
D:\projects\GitTaskBench\queries\Eparse_01\input\Eparse_01_input\
```

and writes the prediction output to:

```
D:\projects\GitTaskBench\output\Eparse_01\output.txt
```

4.4 Step D: Run official grading (Windows encoding fix included)

In **Anaconda Prompt** (env `gtb`), run:

```
cd /d D:\projects\GitTaskBench
set PYTHONIOENCODING=utf-8
set PYTHONUTF8=1
gittaskbench grade --taskid Eparse_01
```

Expected: **Status: PASSED.**

5 Modification(s) and Rationale (Controlled Changes)

I made small but meaningful changes required to reproduce the task on my local Windows setup:

1. **Dependency installation:** Installed `openpyxl` (required by `eparse.core` for Excel parsing) and ensured `pandas` is available for formatting tables.
2. **Output-format alignment:** Adjusted the prediction output formatting (file headers, sheet headers, and DataFrame printing) to match what the official grader recognizes as valid “Excel data blocks”.
3. **Windows grading encoding fix:** The official evaluation script printed Unicode symbols that failed under some default Windows encodings (e.g., GBK). I forced UTF-8 output via:

```
PYTHONIOENCODING=utf-8, PYTHONUTF8=1.
```

Note: These modifications are isolated and measurable because they directly affect whether the official grader detects valid blocks and whether grading can complete successfully on Windows.

6 Results (My Outputs vs Official Evaluation)

6.1 Official output summary (from `gittaskbench grade --taskid Eparse_01`)

The final run achieved **PASSED** with the following reported metrics:

Metric	Value
Filename match rate	100.00%
Content similarity (table_date.xlsx)	100.00%
Content similarity (table_epa.xlsx)	66.67%
Average content similarity	83.33%
Final score (weighted)	88.33%
Status	PASSED

6.2 Artifacts

- Prediction output: `output.txt`
- Official evaluation record: `results.jsonl`
- Screenshot proof of PASS: `grade_pass.png`

7 Debug Diary (Main Blockers and How I Resolved Them)

1. **Repo/task wiring issues:** Early runs failed due to incorrect prompt/output paths (grader could not find valid task prompts or output directories). Fix: confirm the correct prompt directory and output directory, then re-run.
2. **Python dependency/import issues:** `eparse` could not be imported when `PYTHONPATH` was not set correctly, and `openpyxl` was missing. Fix: install missing packages and set `PYTHONPATH` to include the local Eparse repository root.
3. **Grader failure due to Windows console encoding:** JSON printing with Unicode symbols failed under some Windows encodings. Fix: set `PYTHONIOENCODING=utf-8` and `PYTHONUTF8=1`.
4. **Output-format mismatch (“0 Excel data blocks”):** At one point, the grader reported zero detected blocks even when a file existed. Root cause: headers/delimiters did not match the expected structure. Fix: adjust output formatting to match the ground-truth style recognized by the evaluation script.

8 Why This Still Counts as “Agentic”

Although the final pass is produced by running a local script and the official grader, the **development and debugging process** used an LLM agent tool (**Aider**) in a loop:

- Plan: inspect repository structure and identify evaluation expectations.
- Act: propose concrete edits to scripts/config (paths, formatting, encoding).
- Observe: run the grader locally and interpret errors.
- Iterate: revise fixes until the official evaluation passed.

This matches the practical definition of an agentic workflow (multi-step planning + acting with tools + iterative feedback), rather than a single prompt-response.

9 Conclusions

I successfully reproduced GitTaskBench on a single task (Eparse_01) on Windows, passing the official evaluation with a final score of **88.33%**. The primary reproducibility obstacles were environment configuration (dependencies and import paths), output formatting to match evaluation expectations, and Windows console encoding compatibility. These are common failure modes in real-world reproduction work, and the final repo provides a minimal, runnable reproduction package with verifiable artifacts.